Time Complexity: Worked Examples

DSA lecture notes, part 4 (from the YouTube lecture, course “Complete DSA from A to Z”). The lecture is in Hindi/Hinglish; these notes are in simple English. The code is my cleaned-up version of what the teacher shows on screen.

> **Quick recap of the method (from the last video)**
>
> - **Break** a big algorithm into small pieces.
> - Find the complexity of **each piece**, then **add** them.
> - **Drop constants** and the **smaller terms**.
> - A loop counts only if it **depends on the input size n**. For loops, ask: how many times does it run when n grows?
> - **Nested loops multiply**, separate (one after another) loops **add**.

Example 1: Only fixed statements

```
void print() {
    // statement 1
    // statement 2
    // statement 3
    // statement 4
}
```

None of these statements depend on n, so they always run the same number of times. Complexity: **O(1)**, constant.

What if the code has 3 parts that take k1, k2 and k3 steps (all fixed)? Add them: O(k1) + O(k2) + O(k3) = O(1) + O(1) + O(1) = **O(1)**. Constant plus constant is still constant, so **any number of fixed statements is O(1)**.

> **Lesson:** if your algorithm is long, split it into parts, find each part's complexity, then add.

Example 2: Sum of an array

```
int sum = 0;                          // O(1)
for (int i = 0; i < n; i++) {         // runs n times
    sum = sum + arr[i];               // O(1) each time
}
return sum;                           // O(1)
```

The loop runs once for every element. If the array has 4 elements it runs 4 times, if it has 7 it runs 7 times. So the loop is **O(n)**.

Total = O(1) + O(n) + O(1). Drop the small O(1) parts: **O(n)**, linear.

Example 3: Two loops, but the inner one is fixed

```
for (int i = 0; i < n; i++) {          // outer: depends on n
    for (int j = 0; j < 100; j++) {    // inner: always 100
        // O(1) work
    }
}
```

Many people see two loops and write $O(n^2)$ without thinking. **That is wrong here.**

- The outer loop depends on n. If n = 100 it runs 100 times, if n = 200 it runs 200 times.
- The inner loop is **always 100**, whatever n is. It is just a short way of writing 100 statements, so it is a constant: O(1).
- So the total is n × 100, and we drop the constant 100: **O(n)**.

> **Lesson:** do not just count loops. Check **what each loop depends on**. Exam and interview questions usually hide a twist like this.

Example 4: A loop that runs n/2 times

```
for (int i = 0; i < n / 2; i++) {      // n = 100 -> 50 times, n = 200 -> 100 times
    // O(1) work
}
```

This loop runs **n/2** times, so it looks like O(n/2). But 1/2 is just a constant multiplier, and we drop constants. Final answer: **O(n)**.

If there are several parts, find each, add them, and keep only the biggest. For example, a part that is O(n) plus a part that is $O(n^2)$ gives O(n) + $O(n^2)$ = **$O(n^2)$**.

Example 5: Nested loops that both depend on n (or m)

```
for (int i = 0; i < n; i++) {          // outer: n times
    for (int j = 0; j < m; j++) {      // inner: m times for EACH i
        // O(1) work
    }
}
```

For every value of i, the inner loop goes through all its steps (i = 1 → j runs 1..m, i = 2 → j runs 1..m again, and so on). So the work is **multiplied**, not added: **O(n × m)**.

If n and m are equal (as the teacher assumes here), this is **$O(n^2)$**.

> **Rule:** loop inside a loop → **multiply**. Loop after a loop → **add**.

Example 6: Two variables in one loop

```
for (int i = 0, j = n; i < j; i++, j--) {
    // O(1) work
}
```

Here i starts at 0 and goes up, j starts at n and goes down, and the loop stops when they meet. Do a **dry run** with n = 6:

Round	i	j	Condition i < j
1	0	6	true, loop runs
2	1	5	true, loop runs
3	2	4	true, loop runs
4	3	3	false, loop stops

The loop ran 3 times for n = 6, 4 times for n = 8 and 5 times for n = 10. So it runs **n/2** times, which is **O(n)** after dropping the constant.

> **Tip:** when the loop variables move in a tricky way, do a **dry run** with a small n and see how many times it really runs.

Example 7: Bubble sort

Bubble sort sorts an array from small to big. Example: [5, 2, 3, 8, 4] becomes [2, 3, 4, 5, 8]. In each **pass**, it compares neighbours and the biggest remaining element moves to the end.

```
for (int i = 0; i < n - 1; i++) {            // outer: n times
    for (int j = 0; j < n - i - 1; j++) {    // inner: depends on n
        if (arr[j] > arr[j + 1]) {
            // swap them           // O(1)
        }
    }
}
```

The inner loop gets a little shorter in every pass, but it **still depends on n** (if n is 1 million, it runs close to 1 million times in the first pass). Both loops depend on n and one is inside the other, so we multiply: O(n) × O(n) = **$O(n^2)$**.

Dry run for n = 5: pass 1 → inner loop runs 4 times, pass 2 → 3 times, pass 3 → 2 times, pass 4 → 1 time. (My addition: 4 + 3 + 2 + 1 = 10 comparisons, which is n(n−1)/2, and that is still $O(n^2)$ after dropping the constant and smaller term.)

Example 8: Merge sort

Merge sort has 3 steps:

Step	What it does	Complexity
1. Divide	Split the array into two halves, again and again (like binary search, input halves each time).	**O(log n)**
2. Conquer	Sort each half by calling merge sort again (recursion).	Handled by the recurrence below
3. Combine	Merge the two sorted halves into one sorted array. Needs a loop over the elements.	**O(n)**

Because it is recursive, we write a **recurrence relation**: two halves of size n/2 plus the merge work of n.

$$T(n) = 2 \cdot T(n/2) + O(n)$$

You solve this with the **Master theorem** (or the substitution method). The result is **O(n log n)**. The teacher says he will make a separate Master theorem video if viewers ask in the comments.

(My addition, an easy way to remember it: halving the array gives about log n levels, and each level does about n work in total for merging, so n × log n.)

Summary: how to find the complexity of any code

- Understand what the algorithm does, and split it into pieces.
- Ask for every statement or loop: **when n grows, how does the number of times it runs change?**
- Fixed statements → O(1). A loop depending on n → O(n). A loop inside a loop (both depend on n) → multiply.
- If a loop variable moves in a tricky way (n/2, going up and down), do a **dry run**.
- **Add** the complexities of the pieces, then drop constants and smaller terms.
- For recursion, write a recurrence relation and solve it (Master theorem or substitution).

Code	Complexity
Fixed statements	O(1)
Array sum (one loop to n)	O(n)
Outer loop n, inner loop fixed 100	O(n)
Loop running n/2 times	O(n)
Two nested loops, both depend on n	$O(n^2)$
Loop with i going up and j going down	O(n)
Bubble sort	$O(n^2)$
Merge sort	O(n log n)

The teacher ends by asking for more practice questions in the comments. Practice is the key to getting comfortable with this.